

Matías Eduardo Montané Gana

Σ Nota	2,918903903865
Σ Puntaje Tests	6
Σ Puntaje Clean Code	1,42
# Failed E1-BasicCombat	0
# Failed E1-InvalidTeams	0
# Failed E1-RandomBasicCombat	0
# Failed E2-BaseStatsPassives	0
# Failed E2-BeastSkills	0
# Failed E2-DefendAndBreakingPoint	0
# Failed E2-HealingAndQueueSkills	0
# Failed E2-Mix	0
# Failed E2-OffensiveSkills	0
# Failed E2-Random	0
# Failed E3-BasicAttackBoosting	12
# Failed E3-BasicAttackBoostingRandom	30
# Failed E3-BasicPassives	14
# Failed E3-BasicPassivesRandom	30
# Cap. 2	0,45
# Cap. 3	1,45
# Cap. 6	0,22
# Cap. 10	0,1
# MVC	0,2

Clean Code

▼ Cap 2: Meaningful Names 😊 0.45/1.0

Nombre Descriptivo

Los siguientes nombres pueden mejorar:

- <BeastTurnController.cs>:
 - **s** lo usas como abreviacion de skill, siempre evita usar abreviaciones **-0.05**
 - **t** como usaste esta abreviacion mas de 10 veces el descuento es triple **-0.15**
- <CombatManager.cs>:
 - **u** **-0.05**
 - **e** **-0.05**
- <BeastSkill.cs>: **d** como abreviacion de description. Como cometes el error mas de 10 veces el decuento es triple **-0.15**

Revisa tu codigo y evita usar simplemente letras, estas no son descriptivas por lo general

Descuento por tema: **-0.45**

Nombre de Método

No se encuentran métodos cuyo nombre no incluya verbo 

Descuento por tema: **-0.0**

Nombre de Clase y Variable

No se encuentran nombres de variables o clases sin sustantivo 

Descuento por tema: **-0.0**

Encapsular Constantes

Las siguientes constante no están encapsuladas en su propia variable:

- <DamageCalculator.cs>: deberias encapsular el **0.03** y el **1.0** (cuando lo has usa nombres descriptivos) **-0.1**

Descuento por tema: **-0.1**

Una palabra por concepto

Se utiliza una palabra por concepto 

Descuento por tema: -0.0

▼ Cap 3: Functions 1.45/2.0

 **Condición punto base:** No se tiene ningún método de más de 60 líneas.

Cumple condición, tendrás como mínimo 1 punto en el capítulo 

Hacer solo una cosa

Las siguientes funciones realizan más de una cosa:

- <CombatManager.cs>: `HandleHealSkill()` la función lo que hace es manejar la selección de targets y la aplicación de la curación -0.1

Descuento por tema: -0.0

Argumentos

Funciones con más de 3 argumentos

Las siguientes funciones reciben más de 3 argumentos:

- <DamageCalculator.cs>:
 - `CalculatePhysicalDamage` -0.05
 - `CalculateElementalDamage` -0.05
 - `CalculateLastStandDamage` -0.05

Funciones que utilizan out arguments

No se encuentran funciones que reciban out arguments 

Descuento por tema: -0.15

Código Duplicado

No se encuentran bloques de código duplicado 

Descuento por tema: -0.0

Funciones Booleanas

No se encuentran funciones que reciban bool 

Descuento por tema: -0.0

Condiciones en if y while

Se encuentran condiciones que deberían de ser encapsuladas:

- <CombatManager.cs>:
 - en `RemoveBreakingBeastsFromQueue` `u => u is Beast b && b.IsInBreakingPoint && !b.JustRecoveredFromBreakingPoint` -0.05
 - en `HandleTravelerTurn` `!Enum.TryParse<TravelerAction>(input, out var action) || !Enum.IsDefined(action)` -0.05
 - en `ProcessShieldDamage` `shieldsBefore > 0 && target.CurrentShields == 0` -0.05
- <TurnQueueManager.cs>: debes encapsular estas condiciones porque no son legibles
 - `t => (nextRound ? t.IsDefending : t.DefendedLastRound) && !t.IsDead && (!excludeActed || queue == null || !queue.Contains(t))` -0.05
 - `t => t.HasPriorityNextRound && !(nextRound ? t.IsDefending : t.DefendedLastRound) && !t.IsDead && (!excludeActed || queue == null || !queue.Contains(t))` -0.05
 - `!(t.HasPriorityNextRound && (!excludeActed || queue == null || !queue.Contains(t))) && !t.IsDead` -0.05

Descuento por tema: -0.3

Niveles de Indentación

No se encuentran funciones con más de 2 niveles de indentación 

Descuento por tema: -0.0

▼ Cap 6: Objects and Data Structures

0.22/2.0

 Condición punto base: El diseño de las habilidad o de las afinidades respeta el Open-Close Principle.

No se cumple condición 

Polimorfismo:

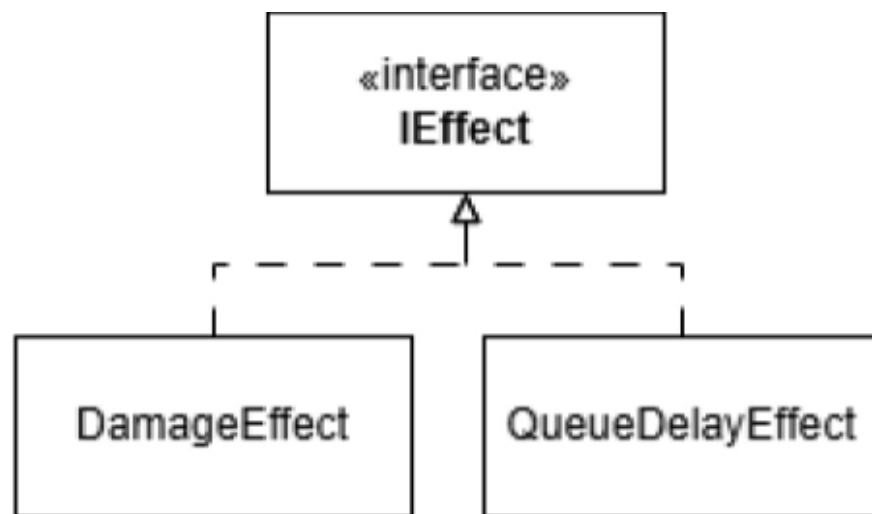
Es necesario que ciertas partes del juego se modelen con polimorfismo.

La modelación de las habilidades del juego debiese haber sido programada utilizando programación orientada a objetos, específicamente herencia y polimorfismo. La idea de esto es que tu puedes crear una abstracción que te permitirá definir de forma genérica como utiliza una habilidad y te permitía añadir nuevas habilidades sin modificar el código que las utiliza ni ninguna clase de tu programa. Para esto es importante usar principio de herencia y polimorfismo.

En tu caso estas representado las habilidades todas en la misma clase

`CombatManager` por medio de metodos como `HandleOffensiveSkill` , `HandleHealSkill` , etc.. En tu codigo estas rompiendo el OCP porque como programaste todo en una unica clase, esto vuelve mas dificil tener que agregar una nueva habilidad. -0.8

Tu modelacion se deberia ver mas o menos de esta forma:



Para esto te recomiendo la implementacion del patron Bridge

Ademas de lo anterior estas usando programacion funcional para manejar la seleccion de objetivos directamente en la clase `CombatManager` por lo que si surgen otras formas de seleccion de objetivos tu programa dificilmente podra adoptarlas. Te recomiendo usar una herencia simple para la modelacion de esto. -0.4

```
if (skill.Target == "Ally")
{
    var result = GetSingleAllyTarget(traveler, _play
```

```

erTeam.Where(t => !t.IsDead).ToList());
    if (result == null) return false;
    targets = result;
}
else
{
    var aliveOthers = _playerTeam.Where(t => !t.IsDead && t != traveler).ToList();
    targets = aliveOthers.Concat(new[] { traveler
}).ToList();
}

```

Descuento por tema: -1.2

Híbridos:

Las clases deben representar un objeto con funcionalidad o una EDD, pero no ambas.

No se encuentran clases híbridas 

Descuento por tema: -0.0

Métodos/Atributos públicos y privados

Los siguientes métodos/atributos tienen un access modifier incorrecto:

- <ActiveSkill.cs> Description -0.01
- <BeastSkill.cs>:
 - Description -0.01
 - Target -0.01
- <Traveler.cs>: ClearCombatStates -0.05

Descuento por tema: -0.08

Train wrecks

Se encuentran los siguientes train wrecks:

- <CombatManager.cs>

- en `StartCombat` `_queueManager.GenerateCurrentRoundQueue().Where(u => !u.IsDead).ToList(); -0.05`
- en `HandleSkillMenu` el metodo `traveler.Skills .Where(name => _activeSkills.FirstOrDefault(s => s.Name == name) is ActiveSkill sk && traveler.CurrentSp >= sk.Sp) .ToList(); -0.05`
- **<TurnQueueManager.cs>:**
 - en `GenerateNextRoundPreview` `_enemyTeam .Where(b => !b.IsDead && b.BreakingPointRoundsRemaining == 1) .Select(b => ((Unit)b, false, _enemyTeam.IndexOf(b))); -0.05`
 - `enemyTeam .Where(b => !b.IsDead && b.JustRecoveredFromBreakingPoint) .Select(b => ((Unit)b, false, _enemyTeam.IndexOf(b))); -0.05`
 - `_playerTeam .Where(t => (nextRound ? t.IsDefending : t.DefendedLastRound) && !t.IsDead && (!excludeActed || queue == null || !queue.Contains(t))) .Select(t => ((Unit)t, true, _playerTeam.IndexOf(t))); -0.05`
 - `_playerTeam .Where(t => t.HasPriorityNextRound && !(nextRound ? t.IsDefending : t.DefendedLastRound) && !t.IsDead && (!excludeActed || queue == null || !queue.Contains(t))) .Select(t => ((Unit)t, true, _playerTeam.IndexOf(t))); -0.05`
 - `_playerTeam .Where(t => !(t.HasPriorityNextRound && (!excludeActed || queue == null || !queue.Contains(t))) && !t.IsDead) .Select(t => ((Unit)t, true, _playerTeam.IndexOf(t))); -0.05`
 - `_enemyTeam .Where(b => !b.IsDead && (nextRound ? b.BreakingPointRoundsRemaining == 0 : (!b.IsInBreakingPoint && !b.JustRecoveredFromBreakingPoint)) && (nextRound ? !b.IsDesprioritized && b.DesprioritizationRoundsRemaining > 1 : !b.IsDesprioritized)) .Select(b => ((Unit)b, false, _enemyTeam.IndexOf(b))); -0.05`
 - `_enemyTeam .Where(b => !b.IsDead && (nextRound ? b.BreakingPointRoundsRemaining == 0 : (!b.IsInBreakingPoint && !b.JustRecoveredFromBreakingPoint)) && (nextRound ? b.IsDesprioritized && b.DesprioritizationRoundsRemaining > 1 : b.IsDesprioritized)) .Select(b => ((Unit)b, false, _enemyTeam.IndexOf(b))); -0.05`
 - `segment .OrderByDescending(x => x.Unit.Stats.Speed) .ThenByDescending(x => x.IsTraveler) .ThenBy(x => x.Index) .Select(x => x.Unit); -0.05`

Descuento por tema: **-0.5**

Ley de Demeter

No se encuentran violaciones a la ley de Demeter 

Descuento por tema: **-0.0**

▼ Cap 10: Classes 0.1/1.5

Las siguientes clases tienen más de una responsabilidad:

- `CombatManager` es una clase que debería manejar la lógica relacionada con la ronda o el turno (a pesar de tener este nombre), si hace más que esto implica que la clase tiene múltiples responsabilidades.

Para empezar la clase maneja directamente las acciones que puede hacer un traveler, es decir atacar, usar una habilidad, defenderse y rendirse. Cada una de estas funciones representa una responsabilidad adicional y provocan que se rompa en OCP porque en caso de querer agregar una nueva acción no puedo extenderlo fácilmente **-0.4**

Respecto a las habilidades, en tu clase tienes el método `ExecuteSkill` que debería estar en otra clase dependiendo de la forma en que estructuras tu código como una fábrica de habilidades.

Además tienes en `CombatManager` todos los métodos que se encargan de la creación de las habilidades lo que claramente debería estar en una clase diferenciada. Dentro de estas están las siguientes donde cada una representa una responsabilidad:

- `HandleOffensiveSkill` **-0.1**
- `HandleHealSkill` **-0.1**
- `HandleReviveParty` **-0.1**
- `ExecuteTargetedHealOrRevive` **-0.1**
- `HandleLegholdTrap` **-0.1**
- `HandleSpearhead` **-0.1**
- `HandleShootingStars` **-0.1**
- `HandleNightmareChimera` **-0.1**
- `CombatView` se encarga de mostrar la mayoría de los mensajes del juego por lo que tiene más de una responsabilidad. Muestra los ataques, mensajes relacionados con las habilidades. Aquí te recomiendo crear una clase para los mensajes de selección de objetivos, ataques, usar polimorfismos para los mensajes relacionados con las habilidades y otra para los menús. **-0.2**

Descuento por tema: -1.4

▼ MVC 0.2/0.5

NOTA: durante este apartado se hablará de proyecto vista, proyecto modelo y proyecto controlador. Sin embargo, no es necesario que sean proyectos distintos, manejarlo en carpetas con namespaces distintos también es válido.

Separación de clases

Se distingue una separación de proyectos modelo/vista/controlador 

Dependencias Incorrectas

Se encuentran las siguientes dependencias incorrectas:

- <Game.cs>:
 - `_view.WriteLine("No se encontraron archivos de equipo.");` -0.05
 - `_view.WriteLine("Elige un archivo para cargar los equipos");` -0.05
 - `_view.WriteLine("Selección inválida o cancelada.");` -0.05
- <CombatManager.cs>:
 - `_combatView.ShowGameState(_playerTeam, _enemyTeam);` -0.05
 - `_combatView.ShowTurnOrder(turnQueue, "Turnos de la ronda");` -0.05
 - `_combatView.ShowTurnOrder(_queueManager.GenerateNextRoundPreview(turnQueue), "Turnos de la siguiente ronda");` -0.05

Descuento por tema: -0.3

▼ Consejos Extras

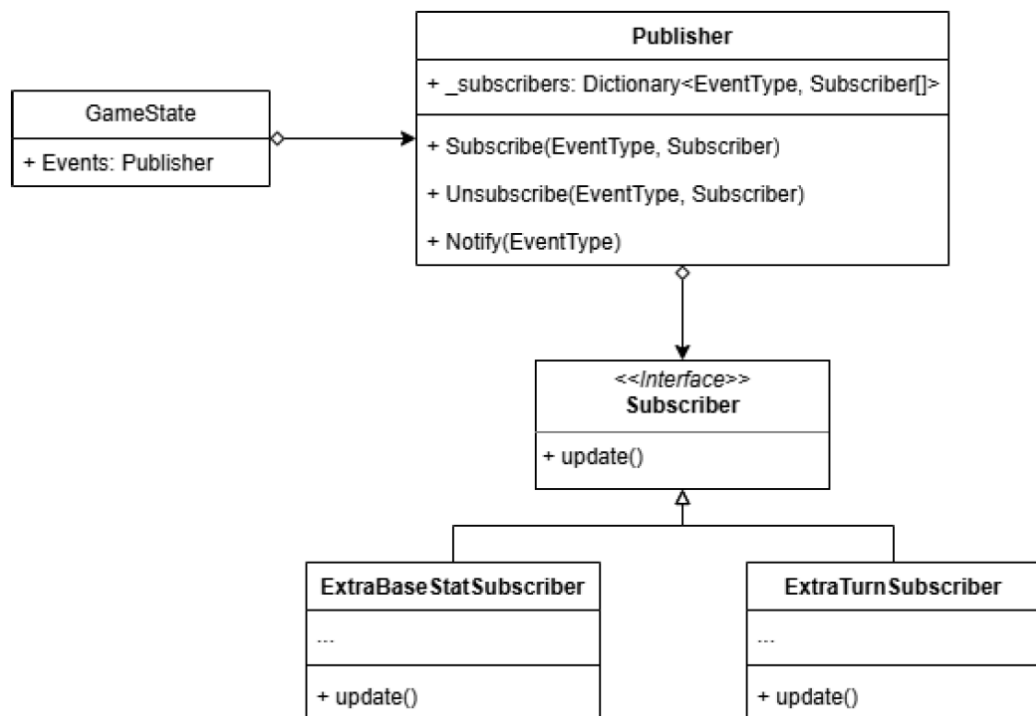
A continuación se presentan algunos consejos adicionales, o anotaciones sobre capítulos que no se evalúan en esta entrega, pero que si se considerarán en la E3.

- esta enum en CombatManager quedaria mejor en su propio archivo

```
private enum TravelerAction { BasicAttack = 1, UseSkill = 2, Defend = 3, Flee = 4 }
```

- Debes usar polimorfismos para la proxima entrega y dividir tu programa en clases y subclases, tienes que dividir mejor las responsabilidades que tienen tus clases
- Para la proxima entrega en relacion con las habilidades pasivas como todas las habilidades pasivas se activan en momentos distintos, además de que no deseamos poner condicionales en todo nuestro código cada vez que se agrega una nueva habilidad. Es necesario poder "notificar" a las habilidades pasivas cuando sucedan dichos eventos.

Para solucionar esta problemática, se propone el patrón de diseño Observer:



Con esto se añadiría a **GameState** un atributo que almacene al **Publisher**, así solo hay que "notificar" al **Publisher** los eventos y este le avisará a las habilidades correspondientes (*Suscribers*).

- tienes mucho que arreglar, te recomiendo empezar esta entrega lo antes posible